```
Output:
Forget it, Donny You're out of your element.
I've always found that writing comes from a great inner pain.
```

We can see that our `useQuoteBot()` function is quite happy with either object type (`LebowskiBot` or `FinkBot`), only the interface matters. We could have declared useQuoteBot() to use the two different classes using a union type declaration (these are covered in great detail in Chapter 3, "Functions"):

```
function useQuoteBot(LebowskiBot|FinkBot $bot): void
```

However, this is not good design. By instead making `useQuoteBot()` dependent on the interface (rather than composite type declaration), we are adhering to an important design principle called *Dependency Inversion* which is one five important OOP design principles known by the acronym SOLID, which we will examine in Chapter 10, "Programming with Objects."

Multiple Interfaces

A class in PHP can only extend one parent (single inheritance), but it can implement multiple interfaces. In the section "Declaring Types" in Chapter 3, "Functions," I showed an argument intersect type declaration of `ArrayAccess&Countable`. `ArrayAccess` and `Countable` are built-in interfaces; if a class implements them, then they are guaranteed to have certain functionality (objects of that class can be counted to an integer, and they can be traversed like an array). Let's do something similar here, but in reverse. We'll create a function that has an object parameter with two interfaces as an intersect type, and a class that satisfies that type declaration.

The function below requires an object with both the `Countable` and `JsonSerializable` interfaces. Like `Countable`, `JsonSerializable` is a built-in interface and specifies a function `jsonSerialize(): mixed` where the return is a data structure that can be processed by PHP's `json_encode()` function:

```
function debugCrew(Countable&JsonSerializable $crew): void
{
    print "\$crew contains " . count($crew) . " officers." . PHP_EOL;
    print json_encode($crew, JSON_PRETTY_PRINT) . PHP_EOL;
}
```